



Programovacie jazyky

Úvod – história, paradigmy

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

22. februára 2024

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

História

Ranné počiatky

- 1837 Charles Babbage navrhol mechanický stroj „**Analytical Engine**“
- 1843 Ada Lovelace do detailov špecifikovala postup výpočtu Bernoulliho čísiel na „Analytical Engine“
 - niektorí historici toto považujú za prvý program
- 1940-50 vznikajú prvé moderné elektricky napájané počítače
 - strojový kód
 - assembler
- 1943 ranný jazyk vyššej úrovne **Plankalkül**, Konrad Zuse, 2D syntax (implementovaný až v roku 1998), inšpiroval Algol, 1958
- 1949 **Short Code**, John Mauchly
 - matematické výrazy sa zapisovali v zrozumiteľnej forme
 - na rozdiel od strojového kódu
 - interpretovaný jazyk $\Rightarrow$ 50 krát pomalší ako strojový kód
- 1952 **Autocode**, Alick Glennie
 - kompilátor pre počítač „Mark 1“ na Manchesterskej univerzite

Fortran (FORmula TRANslation), 1954, John Backus, IBM

- prvý programovací jazyk
 - široko používaný
 - všeobecne použiteľný
 - vyššej úrovne
 - s funkčnou implementáciou (nie len návrh na papieri)
- imperatívny jazyk
- numerické a vedecké výpočty
- používa sa dodnes
- ovplyvnil: Algol, Basic, C, ...

Lisp (LISt Processor), 1958, John McCarthy, MIT

- funkcionálny jazyk
- plne uzátvorkovaná prefixová notácia
 - kód je zoznam
 - manipulácia s kódom ako zoznamom
 - makrá
 - jednoduché parsovanie
- automatická správa pamäte (garbage collection)
- dynamická kontrola typov, iba niekoľko základných typov
- funkcie vyššieho rádu, uzávery (closures), rekurzia
- REPL (read-eval-print loop)
- obľúbený v oblasti umelej inteligencie
- používa sa dodnes (Scheme, Racket, Logo, Emacs Lisp)
- ovplyvnil: Forth, Haskell, ...

Algol (ALGOrithmic Language), 1958

John Backus, Peter Naur, John McCarthy, ..., IBM

- imperatívny jazyk
- pôvodne sa pracovalo s názvom IAL (International Algebraic Language)
- hlavné novinky:
 - blok kódu vymedzený „begin ... end“
 - postupnosť príkazov a deklarácií spojených do bloku
(bez nutnosti vytvoriť explicitne pomenovanú procedúru)
 - vnorené bloky s lexikálnym rozsahom platnosti
 - matematicky presne popísaná syntax jazyka
 - Backus-Naur Form alebo Backus Normal Form (BNF)
 - $\langle \text{expr} \rangle ::= \langle \text{term} \rangle | \langle \text{expr} \rangle \langle \text{addop} \rangle \langle \text{term} \rangle$
- riadiace štruktúry (if, for, switch) $\Rightarrow$ štruktúrované programovanie
- ovplyvnil: C, Pascal (Niklaus Wirth, zjednodušenie Algolu), ...

Cobol (COMmon Business-Oriented Language), 1959

Konzorcium CODASYL (Conference on Data Systems Languages)

- imperatívny jazyk
- stále používaný na udržiavanie starých aplikácií
- najstarší program má Pentagon: MOCAS
(Mechanization of Contract Administration Services)
 - pôvodne asi napísaný v predchodcovi Cobolu: Flow-Matic
autorka: Grace Hopper (dôstojníčka námorníctva USA)
 - zistila, že „business“ zákazníci nemajú radi matematický zápis, preto:
 - anglické slová ako príkazy
 - dlhé mená premenných
 - MULTIPLY HOURLY-RATE BY HOURS-WORKED GIVING GROSS-PAY
- ovplyvnil: PL/1

Apl (A Programming Language), 1960

Kenneth E. Iverson, Harvard University, IBM

- funkcionálny jazyk, hlavný dátový typ: viacrozmerné pole
- program: postupná aplikácia funkcií alebo operátorov na pole
 - niladic (nulárna), monadic (unárna), dyadic (binárna) funkcia
 - všetky primitíva majú rovnakú prioritu a sú sprava asociatívne
 - program sa lepšie číta sprava doľava
 - používa špeciálnu znakovú sadu pre zápis primitív jedným znakom (> 88)
 - vstup: vyžaduje klávesnicu so špeciálnym popisom
 - výstup: terminál tlačiaci na papier špeciálnou guľovou hlavou
- používa sa dodnes
- ovplyvnil: Go, J, Matlab, Wolfram Language, ...

Vyberte 6 rôznych celých pseudonáhodných čísiel z intervalu $\langle 1, 40 \rangle$ a utriedte ich v rastúcom poradí: $x[\uparrow x \leftarrow 6?40]$

- „ $x?y$ “ – „Deal“ – vracia vektor x pseudonáhodných celých čísiel z intervalu $\langle 1, y \rangle$ (rôznych ak $x \leq y$)
 - „ $?x$ “ – „Roll“ – vracia jedno pseudonáhodné celé číslo z $\langle 1, x \rangle$
- „ $x \leftarrow y$ “ – do x priradiť y
- „ $\uparrow x$ “ – vráti utriedujúcu permutáciu indexov vektora x
- „ $x[y]$ “ – vráti vektor prvkov z x podľa indexov z y

<http://tryapl.org/>

Apl – príklad prvočísła

Nájdite všetky prvočísla od 1 po R : $(\sim R \in R \circ . \times R) / R \leftarrow 1 \downarrow \iota R$

- „ ιx ” – „iota” – vracia vektor $1\ 2\ \dots\ x$
- „ $x \downarrow y$ ” – „Drop” – zahodí prvých x prvkov vektora y
- „ $x \leftarrow y$ ” – do x priradiť y
- „ x / y ” – „Replicate” – i -ty prvok y zopakuje $x[i]$ krát
- „ $x \circ . \times y$ ” – vráti maticu, vonkajší súčin x a y
- „ $x \in y$ ” – „Membership” – vracia nula-jednotkový vektor dĺžky $|x|$, pričom na i tom mieste je 1, ak $x[i]$ sa nachádza v y
- „ $\sim x$ ” – „NOT” – vracia vektor, logická negácia x ($0 \leftrightarrow 1$)

<http://tryapl.org/>

Forth, 1968, Charles H. Moore

FOuRTH = 4th (next generation software), IBM 1130 názov súboru max. 5 znakov

- imperatívny jazyk, namiesto premenných používa zásobník
- kompilovaný jazyk majúci aj REPL
- funkcie sa skladajú zreťazením programov (konkatenatívny jazyk)
- kompaktný, ľahko prenositeľný, vhodný pre mikropočítače
 - interpreter nepoužíva gramatiku
 - vstup sa medzerami rozdelí na slová; definícia slova sa vyhľadá v slovníku a vykoná
 - ak sa slovo nenájde, konvertuje sa na číslo a uloží na zásobník
 - definícia nového slova začína slovom „:“ a končí slovom „;“
- vhodný aj pre vstavané systémy
 - implementuje vlastný V/V a multitasking (aj na jednoduchom Atmel AVR)
- používa sa dodnes (Open Firmware boot ROMs)
- ovplyvnil: PostScript, Go, J, Matlab, Wolfram Language, ...

Forth – príklad definovania nového slova FLOOR5

: FLOOR5 (n – n') DUP 6 < IF DROP 5 ELSE 1 - THEN ;

- „:“ – definuj nové slovo „FLOOR5“
- „(“ – komentár až po slovo „)“
- „DUP“ – zdvoj číslo na vrchole zásobníka
- „6“ – vlož 6 na vrchol zásobníka
- „<“ – vyber dve hodnoty zo zásobníka, porovnaj ich a ulož:
 - „-1“ – ak pravda
 - „0“ – ak nepravda
- „IF“ – vyber hodnotu zo zásobníka:
 - „-1“ – vykonaj slová až po „ELSE“ a pokračuj za „THEN“
 - „0“ – preskoč až po „ELSE“ a pokračuj ďalej, ignoruj „THEN“
- „DROP“ – zahod' hodnotu na vrchole zásobníka
- „-“ – vyber 2 hodnoty zo zásobníka, odčítaj ich a ulož výsledok
- `int FLOOR5(int v) return (v < 6) ? 5 : (v - 1); // n – n'`

Forth – příklad REPL

```
> 1 FLOOR5 .
```

```
5
```

```
> 8 FLOOR5 CR .
```

```
7
```

- „.“ – vypíš hodnotu na vrchole zásobníka
- “CR” – vypíš znak nového řádku, t.j. přejdi na nový řádek

<https://replit.com/new/forth>

Prolog (PROgrammation en LOGique – programovanie v logike)

1970, Alain Colmerauer, Philippe Roussel

- deklaratívny jazyk, logické programovanie
- Hornova klauzula: Head :- Body.
 - ak je Body pravda, tak aj Head je pravda.
- program je reprezentovaný reláciami
 - fakty (klauzuly s prázdny telom)
 - cat(tom). $\Leftarrow$ skrátená forma pre: cat(tom) :- true.
 - animal(slon).
 - pravidlá (klauzuly s neprázdny telom): animal(X) :- cat(X).
- výpočet sa spúšťa položením otázky nad reláciami
 - ?- animal(X).
 - X = slon; X = tom
- používa sa dodnes
- ovplyvnil: Clojure, Oz, Erlang, Datalog, ...

Prolog – príklad na vedľajší účinok

```
?- write('Hello world!'), nl.
```

```
Hello world!
```

```
true
```

- „write/1“ – vstavaný unárny predikát pre výpis na obrazovku
 - ako relácia nemá žiadny špecifický význam
 - používa sa iba pre svoj vedľajší účinok (side-effect)
 - vedľajší účinok $\Rightarrow$ „write/1“ nie je čistý (not pure) predikát
- „,/2“ – vstavaný binárny predikát pre konjunkciu
- „;/2“ – vstavaný binárny predikát pre disjunkciu
- „nl/0“ – vstavaný nulárny predikát pre prechod na nový riadok

<http://swish.swi-prolog.org/>

Prolog – príklad na programovanie vyššieho rádu

maplist(_, [], []).

maplist(P, [X|Xs], [Y|Ys]) :-

call(P, X, Y),

maplist(P, Xs, Ys).

neg(0, 1). neg(1, 0).

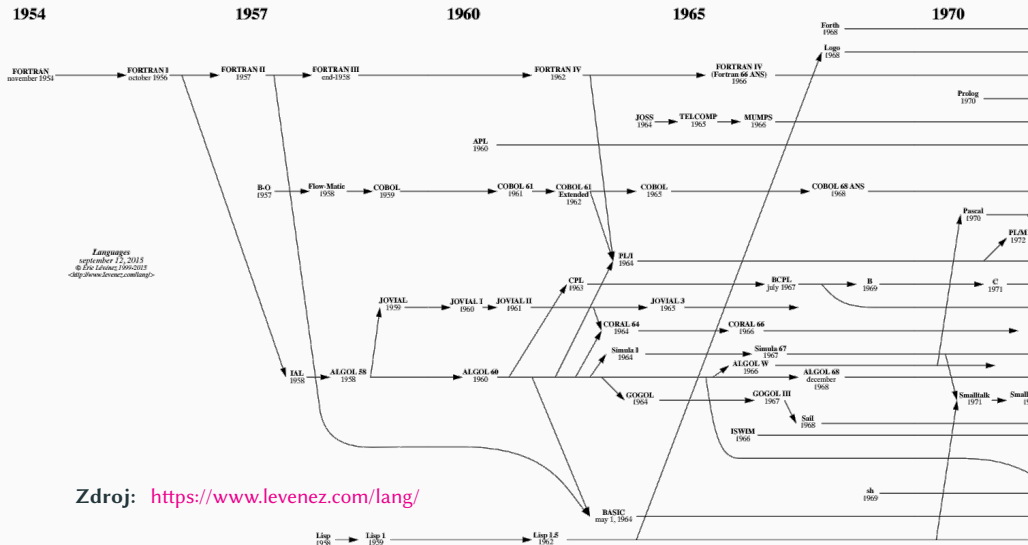
?- maplist(neg, [0, 1, 1], X). $\rightsquigarrow$ X = [1, 0, 0]

?- maplist(neg, [1, X], [X, Y]). $\rightsquigarrow$ X = 0, Y = 1

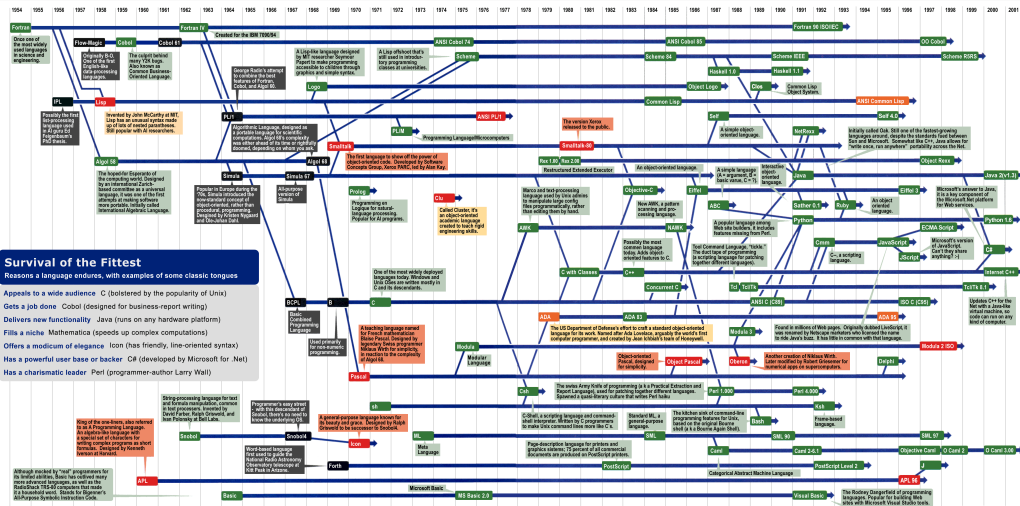
- predikát vyššieho rádu je predikát, ktorý berie iný predikát ako svoj argument (maplist/3, call/3)
 - tým sa dostáva mimo logiku prvého rádu (neumožňuje kvantifikáciu predikátov)
- „call/3“ – vstavaný ternárny predikát
 - $\text{call}(P, X, Y) \Leftrightarrow P(X, Y)$
- ak je predikát P totálna funkcia [t.j. $\forall X \exists! Y: P(X, Y)$], potom:

$$\text{maplist}(P, Xs, Ys) \equiv Ys = \text{map}(P, Xs)$$

Genealógia programovacích jazykov – začiatok



Genealógia programovacích jazykov – výber



Sources: Paul Boutin; Brent Halpern, associate director of computer science at IBM Research; The Retrocomputing Museum; Todd Proebsting, senior researcher at Microsoft; Gjo Wiederholm, computer scientist, Stanford University

Zdroj: http://www.thealmightyguru.com/Wiki/images/6/60/Programming_Language_History.png

Programovacie paradigmy

Základné programovacie paradigmy

Imperatívne programovanie:

- výpočet je postupnosť príkazov
- príkazy menia dáta
- aktualizované dáta predstavujú aktuálny stav programu

Funkcionálne programovanie:

- výpočet je postupnosť vyhodnocovania matematických funkcií
- funkcie vracajú nové dáta (nemodifikujú existujúce dáta)
- nové dáta predstavujú stav programu

Deklaratívne programovanie:

- definuje vlastnosti riešenia bez určenia toku riadenia výpočtu

Imperatívne programovanie

Výpočet je postupnosť príkazov, ktoré priamo menia stav programu.

- premenné obsahujú meniteľné dáta
- príkaz priradenia (modifikujúci obsah premennej)
- globálne premenné

Štýly imperatívneho programovania:

- procedurálne programovanie
 - rozdelenie programu na procedúry, podprogramy, funkcie
 - rekurzia – príbudla až neskôr
 - lokálne premenné, viditeľnosť premenných
 - modifikovanie globálnych premenných $\Rightarrow$ vedľajší účinok (side-effect)
- štruktúrované programovanie
 - kontrola toku riadenia vyššej úrovne, odsadenie kódu
 - while, repeat, for, foreach, ...
 - nepoužívanie goto

Príklady: Pascal, Basic, C, C++, Java, Python, Asembler, ...

Výpočet je postupnosť vyhodnocovania matematických funkcií.

- premenné neobsahujú dáta, iba ich pomenúvajú
- neexistuje príkaz priradenia, dáta sú nemeniteľné
- automatická správa pamäte (garbage collection)
- funkcia môže použiť pre výpočet iba svoje argumenty
 - ⇒ referenčná transparentnosť (rovnaké argumenty → rovnaký výsledok)
 - „efektom“ je iba návratová hodnota ⇒ žiadne vedľajšie účinky
- nové dáta nevznikajú modifikáciou starých, ale môžu sa poskladať z ich častí
 - spoločné časti sa zdieľajú, šetrí sa pamäť
 - dáta sú nemeniteľné ⇒ nevznikne problém pri zdieľaní
- rekurzia namiesto cyklov
- budovanie zložitejších vecí z jednoduchších skladaním funkcií
- funkcia je hodnota 1. kategórie, funkcie vyššieho rádu

Príklady: Clojure, Erlang, Haskell, Elm, Lisp, ...

Objektovo-orientované programovanie

Aplikovateľné na imperatívne aj funkcionálne programovanie.

- každá hodnota je objekt, ktorý je inštanciou nejakej triedy
- triedy pomocou dedenia vytvárajú acyklickú hierarchiu
- objekty komunikujú posielaním správ: príjemca.správa(obsah)

Základné princípy:

- zapúzdrenie
 - skrývanie implementačných detailov
 - vybrané časti vnútorného stavu prístupné len cez get/set metódy
 - abstraktné dátové typy
- polymorfizmus (viac-tvarosť)
 - dynamic dispatch (virtuálne metódy – iba single dynamic dispatch)
 - jednotné spracovanie celej hierarchie (rodiny) objektov
- dedičnosť (zdieľanie spoločného kódu)

Príklady: Common Lisp, Scala, OCaml, C++, C#, Java, ...

Definuje vlastnosti riešenia bez určenia toku riadenia výpočtu.

Riešenie sa hľadá napríklad použitím:

- prehľadávania s návratom (backtracking)
- topologického triedenia
- heuristík

Príklady: Excel, SQL, Regulárne výrazy, Prolog, HTML, ...

Porovnanie výkonu

Vyššia úroveň abstrakcie často prináša pomalší program.

- imperatívny program bez podprogramov je najrýchlejší
 - bez podprogramov $\Rightarrow$ dlhší kód
 - menšia využiteľnosť medzipamäte (> cache miss)
- funkcionálne a procedurálne programovanie
 - často volá funkcie, podprogramy
 - bez „inlining“-u treba zaplatiť cenu volania
- objektovo-orientované programovanie
 - okrem štandardných podprogramov, množstvo get/set metód
 - opäť bez „inlining“-u treba zaplatiť cenu volania
 - dynamic dispatch, tabuľka virtuálnych metód, ďalšie spomalenie
 - vytváranie objektov na halde (heap)
- kompilácia do medzikódu, interpretovaného virtuálnym strojom
 - .NET – CIL, Java – Bytecode, CLISP, Lua, Tcl
- automatická správa pamäte (garbage collection)

Pozri: <https://benchmarksgame-team.pages.debian.net/benchmarksgame/>